

Exercice : Avancer une pose à vitesses constantes

Consigne

Écrire la fonction qui avance une pose de dt secondes à vitesses constantes, linéaire et angulaire. Le programme lit une pose, ses deux vitesses et une durée, et affiche la pose extrapolée. Traiter le cas d'une vitesse angulaire nulle sans diviser par zéro.

Rappel

Le module ne manipule pas des poses nues, mais des poses horodatées, accompagnées de leurs vitesses : linéaire en mètres par seconde, angulaire en radians par seconde.

Connaissant la pose, les vitesses et la durée, on avance : c'est l'intégration d'Euler, la vitesse d'abord, la position ensuite, avec la vitesse déjà mise à jour.

Pour la rotation, on utilise l'axe et l'angle : si la vitesse angulaire est ω (vecteur), sa norme $\|\omega\|$ donne la vitesse de rotation en rad/s. L'angle parcouru en dt est $\theta = \|\omega\| dt$, et l'axe est $\omega/\|\omega\|$.

Fonction d'avance

```
#include <cmath>

// Quaternion : rotation autour d'un axe unitaire.
Quat RotationAxeAngle(const Vec3& axe, float angle)
{
    float h = angle * 0.5f;
    float s = std::sin(h);
    return { std::cos(h), axe.x * s, axe.y * s, axe.z * s };
}

// Quaternion : produit.
Quat Produit(const Quat& a, const Quat& b)
{
    return {
        a.w * b.w - a.x * b.x - a.y * b.y - a.z * b.z,
        a.w * b.x + a.x * b.w + a.y * b.z - a.z * b.y,
        a.w * b.y - a.x * b.z + a.y * b.w + a.z * b.x,
        a.w * b.z + a.x * b.y - a.y * b.x + a.z * b.w
    };
}

// Avance une pose de dt secondes a vitesses constantes.
Pose Avancer(const Pose& pose,
             const Vec3& vitesseLineaire,
             const Vec3& vitesseAngulaire,
             float dt)
{
    // --- Translation ---
    Vec3 nouvellePosition = {
        pose.position.x + vitesseLineaire.x * dt,
        pose.position.y + vitesseLineaire.y * dt,
        pose.position.z + vitesseLineaire.z * dt
    };

    // --- Rotation ---
    float norme = std::sqrt(
        vitesseAngulaire.x * vitesseAngulaire.x +
        vitesseAngulaire.y * vitesseAngulaire.y +
        vitesseAngulaire.z * vitesseAngulaire.z
    );
```

```

);

Quat nouvelleOrientation = pose.orientation;

// Cas d'une vitesse angulaire nulle : pas de division par zero.
if (norme > 1e-6f)
{
    Vec3 axe = {
        vitesseAngulaire.x / norme,
        vitesseAngulaire.y / norme,
        vitesseAngulaire.z / norme
    };
    float angle = norme * dt;
    Quat delta = RotationAxeAngle(axe, angle);
    nouvelleOrientation = Produit(delta, pose.orientation);
}

return { nouvellePosition, nouvelleOrientation };
}

```

Programme complet

```

#include <cmath>
#include <cstdio>

struct Vec3 { float x, y, z; };
struct Quat { float w, x, y, z; };
struct Pose { Vec3 position; Quat orientation; };

Quat RotationAxeAngle(const Vec3& axe, float angle)
{
    float h = angle * 0.5f;
    float s = std::sin(h);
    return { std::cos(h), axe.x * s, axe.y * s, axe.z * s };
}

Quat Produit(const Quat& a, const Quat& b)
{
    return {
        a.w * b.w - a.x * b.x - a.y * b.y - a.z * b.z,
        a.w * b.x + a.x * b.w + a.y * b.z - a.z * b.y,
        a.w * b.y - a.x * b.z + a.y * b.w + a.z * b.x,
        a.w * b.z + a.x * b.y - a.y * b.x + a.z * b.w
    };
}

Pose Avancer(const Pose& pose,
             const Vec3& vl,
             const Vec3& va,
             float dt)
{
    Vec3 p = {
        pose.position.x + vl.x * dt,
        pose.position.y + vl.y * dt,
        pose.position.z + vl.z * dt
    };

    float norme = std::sqrt(va.x * va.x + va.y * va.y + va.z * va.z);
}

```

```

    Quat q = pose.orientation;

    if (norme > 1e-6f)
    {
        Vec3 axe = { va.x / norme, va.y / norme, va.z / norme };
        float angle = norme * dt;
        Quat delta = RotationAxeAngle(axe, angle);
        q = Produit(delta, pose.orientation);
    }

    return { p, q };
}

int main()
{
    Pose pose;
    Vec3 vl, va;
    float dt;

    std::scanf("%f %f %f", &pose.position.x,
                    &pose.position.y,
                    &pose.position.z);
    std::scanf("%f %f %f %f", &pose.orientation.w,
                    &pose.orientation.x,
                    &pose.orientation.y,
                    &pose.orientation.z);
    std::scanf("%f %f %f", &vl.x, &vl.y, &vl.z);
    std::scanf("%f %f %f", &va.x, &va.y, &va.z);
    std::scanf("%f", &dt);

    Pose resultat = Avancer(pose, vl, va, dt);

    std::printf("Position   : %.4f %.4f %.4f\n",
                resultat.position.x,
                resultat.position.y,
                resultat.position.z);
    std::printf("Quaternion: %.4f %.4f %.4f %.4f\n",
                resultat.orientation.w,
                resultat.orientation.x,
                resultat.orientation.y,
                resultat.orientation.z);

    return 0;
}

```

Exemple d'exécution

Pour l'entrée :

```

0.0 0.0 0.0
1.0 0.0 0.0 0.0
1.0 0.0 0.0
0.0 1.5708 0.0
0.5

```

La pose initiale est à l'origine, sans rotation. La vitesse linéaire est (1,0,0) m/s. La vitesse angulaire est (0, $\pi/2$, 0) rad/s, soit une rotation de 90 par seconde autour de y . La durée est 0.5.

La sortie est :

```
Position : 0.5000 0.0000 0.0000
Quaternion: 0.7071 0.0000 0.7071 0.0000
```

La position a avancé de $1 \times 0.5 = 0.5$ mètre selon x . L'orientation a tourné de $(\pi/2) \times 0.5 = \pi/4$ radian, soit 45 autour de y . Le quaternion correspondant est $(\cos 22.5, 0, \sin 22.5, 0) \approx (0.7071, 0, 0.7071, 0)$.

Cas d'une vitesse angulaire nulle

Si la vitesse angulaire est nulle, la norme est nulle. La fonction teste norme $> 10^{-6}$ avant de diviser. Si la norme est inférieure à ce seuil, la rotation n'est pas modifiée : l'orientation reste celle de la pose initiale. Aucune division par zéro n'a lieu.

Exemple. Pour l'entrée :

```
0.0 0.0 0.0
1.0 0.0 0.0 0.0
1.0 0.0 0.0
0.0 0.0 0.0
0.5
```

La sortie est :

```
Position : 0.5000 0.0000 0.0000
Quaternion: 1.0000 0.0000 0.0000 0.0000
```

L'orientation est restée l'identité. Aucune erreur, aucune division par zéro.

À retenir

L'avance d'une pose à vitesses constantes est une intégration d'Euler : la vitesse d'abord, la position ensuite. Pour la rotation, on convertit la vitesse angulaire en axe et angle, puis on applique une rotation delta. Le cas d'une vitesse angulaire nulle est traité par un test de seuil, ce qui évite la division par zéro.

Au-delà de 100, le modèle à vitesses constantes ment plus qu'il n'aide. C'est pourquoi l'extrapolation est bornée à cette durée.